

An AI-Native, Evidence-Traceable Framework for Reservoir Operation Knowledge-Base Construction

Abstract

Reservoir-operation knowledge is distributed across operating manuals, agency records, technical reports, research publications, data documentation, and public-context materials. These sources differ in authority, format, date, purpose, and operational detail. A document-search system can return relevant passages, but it does not by itself preserve the difference between source evidence, conservative evidence organization, and cross-document interpretation.

This paper presents an AI-native framework for constructing a traceable reservoir-operation knowledge base from heterogeneous sources. Here, AI-native means that an agent coordinates semantic decisions across the full construction workflow, while deterministic tools enforce source preservation, schema compliance, artifact contracts, and traceability. The framework has six construction stages: Source acquisition, Evidence extraction, Knowledge consolidation, Synthesis, Indexing, and Report generation. Its central representation has three layers: document-level Evidence Units (EUs), Knowledge Cards (KCs) that organize related EUs around operational questions, and Synthesis Cards (SCs) that make bounded cross-document analyses. Retrieval is treated as a downstream use of the constructed knowledge base, not as a construction stage.

1. Problem and Design Goal

Reservoir operation is governed by interacting physical, hydrologic, institutional, and social conditions. Relevant information may appear in formal release rules, storage targets, drought plans, environmental analyses, forecast products, infrastructure descriptions, agreements, model studies, and event reporting. No single source is guaranteed to state the complete operating context. Different statements may also reflect different dates, scenarios, rule layers, or decision statuses rather than a direct conflict.

The design goal is therefore not merely to summarize a source collection. It is to build a reusable knowledge base in which a reader can distinguish three things: what one source says, how multiple source-grounded findings have been organized, and what broader relationship has been inferred under stated evidence limits. The framework must preserve raw sources before knowledge extraction, provide stable identifiers and locators, validate structured outputs before downstream use, and keep warnings or unresolved issues visible.

The framework is intended for source-grounded reservoir-operation research and knowledge-base development. It does not replace hydrologic modeling, legal interpretation, domain-expert judgment, or the primary source documents themselves.

2. Core Representation

The framework transforms preserved source documents into three record layers. Evidence Units capture document-level operational findings from individual sources. Knowledge Cards conservatively organize EUs that address the same operational question. Synthesis Cards analyze relationships across documents while retaining explicit evidence links and uncertainty. Together, these layers separate what a source states, how related evidence is organized, and what broader relationship is inferred.

Layer	Record	Role	Must preserve
Evidence	Evidence Unit (EU)	One operational finding grounded in one source	Source ID, locator, quote, engineering dimension, confidence
Knowledge	Knowledge Card (KC)	Conservative grouping of EUs that answer the same operational question	Supporting EU IDs, material conditions, disagreements, consolidation type, confidence
Analysis	Synthesis Card (SC)	Evidence-constrained cross-document analysis	Supporting EU IDs, relevant KC IDs, pattern, evidence depth, confidence, source-checking context

A completed Lake Powell run shows the intended separation using direct excerpts from `runs/lake_powell_20260729_kb/`.

Layer	Example
Source	RES-020: “In 2005 discharges were affected because reservoir elevations were lower than at any other time since initial filling, as shown in Figure 4-21, and low DO water in the metalimnion was located just above the penstock elevation (Figure 1-2).”

Layer	Example
EU	EU-LP-RES-020-07: In most years the metalimnetic oxygen minimum does not affect dam discharges because it sits well above the penstock elevation (3,470 ft), but in 2005 reservoir elevations were the lowest since initial filling and the low-DO metalimnetic water was located just above the penstock elevation, allowing oxygen-depleted water to pass through the dam.
KC	KC-LP-UNC-10: In fall 2005, a plume of oxygen-depleted water in Lake Powell was captured in the penstock withdrawal zone and exported through Glen Canyon Dam, dropping tailwater dissolved oxygen below 4.0 mg/L in September 2005 and violating EPA cold-water fishery DO criteria (1-day mean minimum 4.0 mg/L, 7-day mean minimum 5.0 mg/L, 30-day mean 6.5 mg/L) (RES-020-02). This occurred because in most years the metalimnetic oxygen minimum sits well above the penstock elevation (3,470 ft) and does not affect dam discharges, but in 2005 reservoir elevations were the lowest since initial filling, placing the low-DO metalimnetic layer just above the penstock elevation and allowing oxygen-depleted water to pass through the dam (RES-020-07).
SC	SC-LP-011: In September 2005, Glen Canyon Dam tailwater dissolved oxygen fell below EPA cold-water-fishery criteria because historically low reservoir elevations placed the reservoir's low-oxygen metalimnetic layer directly at the penstock withdrawal elevation, allowing oxygen-depleted water to be exported through the dam – a documented water-quality failure mode tied specifically to low reservoir elevation.

This completed-run example shows the intended separation. The Source row keeps source wording. The EU row records a single-source finding with an identifier and locator. The KC row organizes related EUs around an operational question while keeping supporting evidence visible. The SC row states a bounded cross-document analysis with explicit EU/KC support. This separation keeps evidence, organization, and analysis reviewable as distinct objects.

Every record must resolve back to preserved source evidence. Let $\mathcal{S}$ be the preserved source corpus, E the set of EUs, K the set of KCs, and A the set of SCs. The global traceability constraint is:

$$\forall x \in (E \cup K \cup A), \quad \text{Traceable}(x)$$

In practice, traceability means that a reviewer can follow a record’s IDs and locators back to the preserved source text and evidence quote. The same traceability definition is used throughout construction, validation, indexing, and reporting.

Synthesis Cards use six primary patterns when the evidence supports them: `decision_process`, `constraint_structure`, `operational_tradeoff`, `operating_regime_change`, `historical_operation_failure`, and `operational_consequence`. Each SC has exactly one primary pattern. Secondary lenses may be recorded when they are supported, but the workflow does not fill all possible analytical categories merely to make the output look complete.

A source discrepancy or evidence gap is not itself a synthesis pattern. If sources disagree about the same object under the same date, definition, status, scenario, and decision context, the issue belongs in a `contested_or_unresolved` KC or a validation warning. Differences that arise from different forecast dates, proposed versus final rules, or baseline versus emergency operations should be preserved as context. They should not be turned into a higher-level conclusion before the evidence supports that move.

3. Six-Stage Construction Workflow

The routine workflow has six construction stages. Run orchestration starts a compliant run, enforces stage order, and supports handoff, but it is not a seventh knowledge-production stage.

At run level, the workflow is a sequence of typed transformations:

$$\begin{matrix} f_E & f_K & f_A & f_I & f_R \\ S \rightarrow E \rightarrow K \rightarrow A \rightarrow I \rightarrow R \end{matrix}$$

Here I denotes index-ready records and R denotes the generated report and claim-evidence map. Each transition is allowed only after the upstream stage passes its required automated checks:

$$f_{t+1}(Y_t) \text{ is admissible only if } V_t(Y_t) = 1$$

For source acquisition, automated validation checks schema conformance and acquisition status. For later stages, it checks schema conformance and traceability integrity. These checks are narrow by design. They confirm that records are well-formed and that cited IDs resolve; they do not certify that a semantic judgment is correct.

Stage	Primary input	Primary output	Purpose
1. Source acquisition	Reservoir scope and source leads	Preserved source corpus	Build a reviewable evidence base before extraction

Stage	Primary input	Primary output	Purpose
2. Evidence extraction	Preserved source text	Evidence Units	Create document-level, source-grounded operational findings
3. Knowledge consolidation	Validated EUs	Knowledge Cards	Organize related EUs by operational question
4. Synthesis	Validated EUs and KCs	Synthesis Cards	Analyze supported cross-document relationships
5. Indexing	Validated EUs, KCs, and SCs	Encoded index records	Prepare records for search or embedding backends
6. Report generation	Validated structured records	Report and claim-evidence map	Render the knowledge base into readable prose

The authoritative artifact contracts are maintained in `AGENTS.md`, `docs/run-artifact-contract.md`, and `docs/validation-framework.md`. The subsections below explain the methodological role of each stage rather than repeating every file-level contract.

3.1 Source Acquisition

Source acquisition builds the evidence corpus before any knowledge claim is created. It includes source discovery, candidate screening, download or local preservation, text extraction, quality review, and inventory validation. The stage is deliberately called *source* acquisition rather than data acquisition: it preserves documents, webpages, and data documentation as evidence sources. A separate structured-data intake contract would be needed for time series, APIs, or other operational datasets.

Sources are reviewed by role rather than treated as interchangeable. The workflow distinguishes official or operating-authority sources, research or technical sources, and public or contextual sources. These families are not quality scores. Source importance, content quality, accessibility, and readiness for EU extraction are recorded separately. Parser fallbacks, HTML responses from PDF URLs, paywalls, and short text extractions are retained as review signals.

The output is a preserved corpus, not a claim set. Its core artifacts include a source manifest, candidate inventory, acquired-source inventory, raw files, extracted text, and parsing notes.

3.2 Evidence Extraction

Evidence extraction processes each source independently. Long documents are split into page-, section-, or chunk-aware packets so that one source can yield multiple distinct findings. The result is an EU: a concise operational finding with a quote and locator that allow a reviewer to return to the source.

An EU is accepted only when it is document-level, operationally useful, self-contained, and traceable. It preserves an EU ID, source ID, title or URL, locator, evidence quote, confidence, extraction method, engineering dimension, and notes. Cross-document statements are excluded at this stage.

The engineering dimension helps organize evidence without replacing the source wording. Example dimensions include Operation Rules, Storage Capacity and Storage Targets, Inflow Forecast, Observation and Data, Modeling, Regulation / Governance, and Emergency Operations. Exact or semantic duplicates within the same source may be removed, but distinct rules, thresholds, dates, facilities, and conditions remain separate EUs.

3.3 Knowledge Consolidation

Knowledge consolidation creates KCs by grouping EUs that answer the same operational question. Engineering dimension can provide a first view, but the actual grouping key is the question being answered. EUs that share a broad topic but describe different rules, scenarios, facilities, dates, statuses, or decision contexts remain in separate cards.

Each KC uses one consolidation type: `repeated_fact`, `complementary_facts`, or `contested_or_unresolved`. A KC cites every supporting EU and keeps material qualifications visible. It may show that an issue is unresolved, but it does not resolve an apparent conflict by inference.

This stage is useful because many reservoir-operation questions require more than one source-grounded statement but still do not justify analysis. For example, separate EUs about forecast inputs, seasonal targets, and downstream requirements can be organized under one question about release determination. That KC prepares the evidence for synthesis while keeping the evidentiary boundary intact.

3.4 Synthesis

Synthesis begins only after EUs and KCs have passed validation. KCs help locate related operational questions. EUs and their locators provide the evidence chain. A Synthesis Card is created only when the result is more than a restatement of one EU or KC and is supported by the cited evidence.

For important, technical, numerical, legal, historical, or contested claims, the workflow uses a deeper source-checking pattern:

EU/KC -> candidate relationship -> reread original source context -> Synthesis Card

The card records its primary pattern, any supported secondary lenses, supporting EU IDs, relevant KC IDs, confidence, next action, and evidence depth. Source-checked cards also record which source context was reread and whether that context confirmed, narrowed, complicated, or rejected the candidate relationship.

Evidential support is judged against the cited EU and KC sets, not against the fluency of the generated statement. Automated validation can confirm that the cited IDs resolve. It cannot confirm that the cited evidence semantically entails the analysis. That judgment is made during synthesis and can be sampled through human review.

3.5 Indexing

Indexing is an offline construction stage. It transforms validated EU, KC, and SC records into index-ready records while retaining stable IDs, record type, metadata, locators, confidence, and evidence-chain fields. The stage may prepare structured text, embeddings, keyword fields, or a hybrid index for a selected backend.

Indexing does not create new knowledge claims. Each index record wraps exactly one underlying EU, KC, or SC. It can make a record searchable, but it cannot make an untraceable record traceable or change the boundary between evidence, organization, and analysis.

Retrieval is a downstream use of the index. Query execution, ranking, and retrieval evaluation can test whether the constructed knowledge base returns adequate evidence for operational questions. Such testing evaluates usefulness; it does not alter the constructed records.

3.6 Report Generation

Report generation is a controlled transformation of validated records into a human-readable artifact. A report can explain operation purposes, rules, storage conditions, emergency actions, governance, data and models, tradeoffs, and evidence limitations. It must not introduce a material claim that cannot be traced through its claim-evidence map.

Report prose distinguishes source-grounded findings from synthesis-level interpretation. Material claims cite EUs, KCs, SCs, or source records as appropriate. Evidence gaps, uncertainty, parser issues, and unresolved questions remain visible instead of being converted into confident narrative statements.

4. AI-Native Workflow Execution

The framework is AI-native in its execution, not only in its use of generated text. The agent coordinates the full construction workflow: it selects the stage protocol, interprets source

context, screens semantic relevance, drafts EUs, consolidates KCs, creates SCs, coordinates repairs, and writes controlled report prose.

Deterministic tools handle the parts of the workflow that should not depend on model judgment. They preserve files, extract text, create packets, validate JSON and JSONL, render inspection views, check ID traceability, and enforce stage artifact contracts.

This execution design makes the agent a semantic operator rather than a single all-purpose generator. The detailed implementation architecture is included in Appendix A. The trust boundary created by this division is discussed with validation and human review in the next section.

5. Validation, Human Review, and Trust Boundary

Validation is distributed across the workflow rather than treated as a seventh construction stage. Every stage completes automated validation before its output is used downstream. A stage manifest and stage summary record inputs, outputs, warnings, unresolved issues, cleanup status, downstream readiness, and the validation result.

Automated validation is deliberately narrow. It checks schema conformance and ID traceability. For source acquisition, it also checks acquisition status. It does not confirm that a cited EU entails the statement built on it, that a KC's grouping was the best judgment call, or that an SC's relationship is the most reasonable reading of the evidence.

This is the framework's trust boundary. The agent is trusted to perform semantic labor, but it is not trusted as an evidence authority. Deterministic validation is trusted to check record shape and resolvable references, but it is not trusted to certify semantic correctness. Evidence authority remains with the preserved sources and the traceable record chain.

Human review supplies that semantic check. It is a documented sampled evaluation activity, available for every stage but not a routine blocking gate unless a particular run or study requires it. EU review addresses faithfulness, relevance, and value. KC review addresses faithfulness, consolidation appropriateness, and value. SC review addresses faithfulness, reasoning soundness, and value. Report review addresses faithfulness, readability, and value. Indexing may be evaluated through retrieval questions after construction.

This separation matters because construction and evaluation answer different questions. Construction asks whether records and evidence chains have been created according to the contract. Evaluation asks whether those records are useful for an intended task, such as finding evidence for an operational question or supporting a readable case-study report. Evaluation can motivate a later revision, but it should not silently rewrite the validated record layer.

Because the trust boundary is explicit, the agent layer is replaceable. A different model, prompt, or execution backend may coordinate the same stage, but the output still has to be

schema-conformant, traceable, and reviewable. The method therefore depends less on a particular model identity than on the combination of preserved evidence, structured records, deterministic checks, and sampled semantic review.

6. Implementation Example

The framework has been applied to a Lake Powell knowledge-base run stored under `runs/lake_powell_20260729_kb/`. The canonical run artifacts record 201 candidate source records and 66 acquired source records. They also record 442 Evidence Units, 81 Knowledge Cards, 15 Synthesis Cards, 538 index-ready records, and a report claim-evidence map with 30 mapped claims.

The run-level contract validation passed. Stage validation also illustrates why warnings are kept visible rather than treated as cosmetic noise. Source acquisition completed with warnings about acquisition or source-quality issues, and evidence extraction recorded one warning. The later knowledge consolidation, synthesis, indexing, and report-generation stages passed schema and traceability checks. In other words, the framework did not present the run as perfect. It preserved source limitations while still allowing validated downstream records to be inspected and used.

This example is not a hydrologic validation of Lake Powell operations. It is an implementation check of the construction method: the workflow can preserve sources, build the three record layers, encode them for retrieval, and produce a report while keeping record counts, validation status, and evidence paths visible.

7. Workflow Refinement and Transfer

The framework is intended to improve through documented workflow iteration. A well-documented reservoir can serve as a development case because it exposes failure modes that are hard to anticipate from the abstract design alone: missing source families, weak candidate screening, shallow extraction from dense operating documents, overbroad EUs, inappropriate KC grouping, unsupported synthesis, weak source rereading, broken evidence navigation, or report claims that are technically traceable but semantically too strong.

These failures should be treated as signals about the workflow, not only as mistakes in individual records. A missed class of operating rules may indicate that source-discovery queries or source-family definitions need revision. A pattern of overbroad EUs may require a sharper extraction prompt, more section-aware packetization, or new anti-pattern examples. Repeated KC grouping errors may show that the operational-question criterion is under-specified. Unsupported SCs may require stricter source-rereading triggers or a clearer definition of the six synthesis patterns. Report faithfulness issues may point to stronger claim-evidence-map requirements.

Prompt iteration is therefore part of the method. The prompts are not tuned to make one run look better; they are revised to make stage boundaries easier for the agent to follow and easier for validators and reviewers to inspect. A useful prompt revision should state the failure mode it addresses, the stage it affects, the expected behavioral change, and the validation or review signal that will show whether it helped. Schema changes should be made only when the record contract needs to capture evidence that the current schema cannot represent, not merely to accommodate malformed outputs.

Transfer to another reservoir tests whether the refined workflow remains usable in a different operational and documentary setting. This is workflow transfer validation, not model-parameter training or a conventional machine learning train/test split. The expected invariants are the EU/KC/SC layer boundaries, preservation of source evidence, stage validation before downstream use, and explicit separation between automated checks and semantic review. The expected variables are reservoir-specific source availability, governing institutions, operational regimes, terminology, data products, and the synthesis patterns that are actually supported by the evidence.

The central transfer question is not whether two reservoirs produce the same number of records or the same pattern distribution. It is whether the same construction logic can still produce inspectable, traceable, and useful records under a new reservoir context, and whether any needed prompt or protocol revisions are motivated by documented failure modes rather than ad hoc rewriting after the fact.

8. Contributions and Limitations

The framework contributes four main elements. First, it defines a three-layer EU/KC/SC representation that separates evidence, organization, and analysis. Second, it provides a six-stage construction workflow that can be executed and audited. Third, it divides responsibility between an agent and deterministic scripts, using each where it is strongest. Fourth, it separates automated validation from semantic human review.

The limitations are equally important. Source coverage is bounded by the collected corpus. Extraction quality depends on preservation and parsing quality. Synthesis remains a constrained analytical activity and requires source rereading for consequential claims. The framework therefore records gaps, warnings, parser fallbacks, and unresolved issues rather than treating apparently complete output as complete operational knowledge.

9. Conclusion

An AI-native reservoir-operation knowledge base should be more than a set of embeddings or generated summaries. It should preserve evidence before interpretation, make each

transformation explicit, and keep the path from report or index record back to source text available for review.

The six-stage framework operationalizes those requirements through Source acquisition, Evidence extraction, Knowledge consolidation, Synthesis, Indexing, and Report generation. Its EU, KC, and SC layers provide a disciplined basis for later retrieval, analysis, and reporting without collapsing those uses into the evidence itself.

Appendix A: Agent Architecture, Reproducibility Contract, and Operating Prompts

The main text describes the method. The repository carries the executable contract. The routine run requirements are maintained in:

Contract	Role
AGENTS.md	Project-level workflow rules and required run artifacts
docs/run-artifact-contract.md	Stage manifests, summaries, validation files, and cleanup rules
docs/validation-framework.md	Automated validation and sampled human-review design
skills/<stage>/SKILL.md	Stage-specific operating instructions
schemas/ and validation/	Machine-readable schemas and validation scripts

Each numbered stage folder should contain its canonical records, a `stage_manifest.json`, a `stage_summary.md`, `automated_validation.json`, and `human_review.json`. Stages 2 through 4 also include review navigation files so that EUs, KCs, and SCs can be checked against preserved source text.

A.o Agent Architecture Components

The reservoir knowledge-base agent is not a single prompt. It is a coordinated workflow made from project rules, stage-specific skills, deterministic scripts, schemas, validators, and durable artifacts.

Component	Function
Project instructions	Define the six-stage workflow, layer boundaries, run rules, and required artifacts
Run orchestration skill	Starts or resumes a run, enforces stage order, prepares handoff checks, and validates the final

Component	Function
	run contract
Stage skills	Provide the operating protocol for source acquisition, evidence extraction, knowledge consolidation, synthesis, indexing, and report generation
Agent reasoning	Performs semantic screening, EU drafting, KC grouping, SC analysis, repair decisions, and controlled report writing
Deterministic scripts	Preserve files, parse text, create extraction packets, render Markdown views, build index records, and validate artifacts
Schemas	Define the required JSON/JSONL structure for sources, EUs, KCs, SCs, index records, reviews, and claim-evidence maps
Validation tools	Check schema conformance, acquisition status, traceability integrity, run-contract completeness, and report claim coverage
Stage artifacts	Store canonical records, manifests, summaries, validation results, and human-review workpapers
Preserved source corpus	Provides the evidence authority for all downstream records and reports
Human review workpapers	Support sampled semantic review without making routine construction depend on manual approval

At execution time, the agent reads the project instructions, invokes the appropriate stage skill, uses scripts where deterministic processing is available, writes canonical records, runs validators, repairs failures, and only then moves to the next stage. Each new reservoir run is written to a new `runs/<reservoir_slug>_<run_date>/` folder, so historical runs are preserved.

The prompts below are condensed operating prompts. They are not a replacement for the repository skills and validators. They preserve the practical instructional core of the six routine stages so the workflow can be described or reproduced outside the local agent environment.

A.1 Source Acquisition Prompt

Task: Build the preserved source corpus for `<reservoir_name>`.

Reservoir scope:

- `reservoir_id`: `<reservoir_id>`
- `reservoir_id_system`: `<reservoir_id_system>`

- reservoir_name: <reservoir_name>
- basin/operator/context: <scope_notes>

Priority collection themes:

- authorized purposes, operating objectives, and multi-objective tradeoffs;
- storage capacity, elevation targets, minimum pools, guide curves, and storage-protection rules;
- release rules, seasonal rules, thresholds, triggers, operating tiers, and decision criteria;
- infrastructure constraints, outlet/intake capacity, hydropower facilities, conveyance constraints, and maintenance or physical operating limits;
- inflow forecasts, forecast horizons, uncertainty ranges, hydrologic scenarios, and drought/flood outlooks;
- observation systems, monitoring data, reservoir variables, official data portals, and data documentation;
- reservoir models, simulation studies, scenario methods, assumptions, and decision-support tools;
- laws, agreements, regulations, consultation processes, agency authority, and governance responsibilities;
- drought, flood, emergency, real-time, or special-event operations;
- environmental, ecological, water-quality, sediment, temperature, or habitat constraints that affect operations;
- stakeholders, water users, tribes, hydropower interests, downstream communities, and public-context materials;
- historical operating events, failures, shortages, rule changes, controversies, and documented consequences.

Instructions:

1. Discover a broad candidate pool from official operating authorities, government repositories, agency reports, research indexes, citation chains, data documentation, and credible public-context sources.
2. Screen candidates before download using title, URL/DOI, source type, metadata, abstract, and light inspection. Record source-family hints separately from quality judgments.
3. Preserve selected sources locally as raw PDF, HTML, text, or landing-page files. Assign stable source IDs. Record redirects, paywalls, failed access, short extraction, parser issues, and recovery attempts.
4. Extract readable text. Use robust plain-text extraction first and structured parsing when available. Keep raw files before extraction.
5. Review final source family, document type, importance, content quality, accessibility, and EU-readiness.
6. Validate source manifest, candidate inventory, acquired-source inventory, file paths, statuses, and acquisition warnings.

Outputs:

- 01_source_acquisition/source_manifest.json
- 01_source_acquisition/sources/candidate_inventory.jsonl
- 01_source_acquisition/sources/source_inventory.jsonl
- 01_source_acquisition/sources/raw/
- 01_source_acquisition/sources/text/

- 01_source_acquisition/automated_validation.json
- 01_source_acquisition/human_review.json
- 01_source_acquisition/stage_manifest.json
- 01_source_acquisition/stage_summary.md

Boundary:

This stage creates a preserved evidence corpus. It does not create Evidence Units, Knowledge Cards, Synthesis Cards, index records, or report claims.

A.2 Evidence Extraction Prompt

Task: Extract document-level Evidence Units from the validated source corpus.

Inputs:

- runs/<run_id>/01_source_acquisition/sources/source_inventory.jsonl
- runs/<run_id>/01_source_acquisition/sources/text/

EU evidence topics / engineering dimensions (current schema uses 13):

- Operation Purposes
- Multiple Objectives
- Storage Capacity and Storage Targets
- Operation Rules
- Emergency Operations
- Real-Time Operations
- Regulation / Governance
- Uncertainty and Risk Management
- Observation and Data
- Inflow Forecast
- Modeling
- Stakeholders
- Operation Failure

Instructions:

1. Confirm that the source inventory and extracted text files exist and are ready for EU extraction.
2. Process each source independently. Split long documents into page-, section-, or chunk-aware packets. Do not rely on a truncated document prefix.
3. Draft candidate EUs only for operationally useful, document-level findings.
4. Each EU must be concise, self-contained, traceable, and supported by a short evidence quote and locator.
5. Assign one primary engineering dimension using source wording and the approved dimension taxonomy.
6. Remove exact or semantic duplicates within a source when they state the same operational fact. Keep separate rules, thresholds, facilities, dates, scenarios, and conditions as distinct EUs.
7. Exclude cross-document conclusions, generic background, navigation text, and unsupported summaries.
8. Validate the EU JSONL. Repair records on validation failure; do not loosen the schema.

Outputs:

- 02_evidence_extraction/evidence_units.jsonl
- 02_evidence_extraction/evidence_units.md
- 02_evidence_extraction/automated_validation.json
- 02_evidence_extraction/human_review.json
- 02_evidence_extraction/source_navigation.md
- 02_evidence_extraction/stage_manifest.json
- 02_evidence_extraction/stage_summary.md

Boundary:

An EU is one source-grounded operational finding. It is not a cross-document conclusion and not a synthesis claim.

A.3 Knowledge Consolidation Prompt

Task: Consolidate validated Evidence Units into Knowledge Cards.

Inputs:

- runs/<run_id>/02_evidence_extraction/evidence_units.jsonl
- runs/<run_id>/02_evidence_extraction/automated_validation.json

KC organization topics / engineering dimensions (same 13-topic taxonomy as EUs):

- Operation Purposes
- Multiple Objectives
- Storage Capacity and Storage Targets
- Operation Rules
- Emergency Operations
- Real-Time Operations
- Regulation / Governance
- Uncertainty and Risk Management
- Observation and Data
- Inflow Forecast
- Modeling
- Stakeholders
- Operation Failure

KC consolidation types:

- repeated_fact: multiple EUs state the same operational finding;
- complementary_facts: EUs answer the same operational question from different source details, conditions, or perspectives;
- contested_or_unresolved: EUs preserve a real unresolved disagreement or uncertainty that should not be resolved by inference.

Instructions:

1. Confirm that EU validation has passed or that warnings have been recorded and accepted for downstream use.
2. Group EUs first by engineering dimension as a working view, then by the operational question they answer.

3. Create a KC only when EUs express a repeated fact, complementary facts, or a meaningfully contested or unresolved point.
4. Do not merge EUs merely because they share a broad topic. Preserve separate rules, thresholds, facilities, dates, scenarios, statuses, and decision contexts when they change the operational meaning.
5. Cite every supporting EU ID. Preserve source-defined names, values, units, dates, rule contexts, and qualifications.
6. Record the consolidation type, confidence, coverage note, and unresolved disagreement when applicable.
7. Validate Knowledge Cards before Synthesis and render the Markdown reading view.

Outputs:

- 03_knowledge_consolidation/knowledge_cards.jsonl
- 03_knowledge_consolidation/knowledge_cards.md
- 03_knowledge_consolidation/automated_validation.json
- 03_knowledge_consolidation/human_review.json
- 03_knowledge_consolidation/source_navigation.md
- 03_knowledge_consolidation/stage_manifest.json
- 03_knowledge_consolidation/stage_summary.md

Boundary:

A KC organizes evidence around an operational question. It is not new source evidence, not a recommendation, and not a cross-document analytical conclusion.

A.4 Synthesis Prompt

Task: Create source-grounded Synthesis Cards from validated EUs and KCs.

Inputs:

- runs/<run_id>/02_evidence_extraction/evidence_units.jsonl
- runs/<run_id>/03_knowledge_consolidation/knowledge_cards.jsonl

SC synthesis topics / primary patterns:

- decision_process: how information, rules, authority, judgment, and system conditions translate into an operating decision or implementation;
- constraint_structure: how physical, operational, legal, ecological, infrastructural, or institutional constraints define the feasible operating space;
- operational_tradeoff: how competing objectives, risks, or stakeholder outcomes are connected through the same operation or constrained resource;
- operating_regime_change: how rules, objectives, constraints, capabilities, or decision environments change across time or operating regimes;
- historical_operation_failure: a documented episode in which an operational function, requirement, or target was not achieved;
- operational_consequence: an observed, modeled, or projected consequence of an identified operation, rule, decision, or operating regime.

Instructions:

1. Confirm that EU and KC validation has passed.
2. Use KCs to identify related operational questions and supporting EUs. Do not treat a KC as a substitute for the original source context.
3. Draft a candidate operational relationship only when it is more than a restatement of one EU or KC.
4. Reread original source passages when the claim is important, technical, quantitative, causal, legal, historical, contested, or sensitive to date, scenario, status, version, or decision context.
5. Create one SC with exactly one primary pattern:
decision_process, constraint_structure, operational_tradeoff, operating_regime_change, historical_operation_failure, or operational_consequence.
6. Include an explicit scope, supported analysis chain, evidence role map, EU/KC evidence links, uncertainty or exceptions, operational implication, confidence, next action, and evidence depth.
7. For source-checked cards, record source locator summary and a source verification note explaining what rereading confirmed, narrowed, complicated, or rejected.
8. Do not use SCs for generic source gaps or unresolved discrepancies. Those belong in KC notes, validation warnings, or research notes.
9. Validate Synthesis Cards before Indexing or Report generation.

Outputs:

- 04_synthesis/synthesis_cards.jsonl
- 04_synthesis/synthesis_cards.md
- 04_synthesis/automated_validation.json
- 04_synthesis/human_review.json
- 04_synthesis/source_navigation.md
- 04_synthesis/stage_manifest.json
- 04_synthesis/stage_summary.md

Boundary:

An SC is an evidence-constrained analytical record. It must cite supporting EUs and, where useful, KCs. It does not create new source evidence.

A.5 Indexing Prompt

Task: Encode validated EUs, KCs, and SCs into index-ready records.

Inputs:

- runs/<run_id>/02_evidence_extraction/evidence_units.jsonl
- runs/<run_id>/03_knowledge_consolidation/knowledge_cards.jsonl
- runs/<run_id>/04_synthesis/synthesis_cards.jsonl

Instructions:

1. Confirm validated EU, KC, and SC JSONL files exist.
2. Create exactly one index record for each selected knowledge-base record.
3. Build concise index text from the record content and material context.
4. Preserve record type, stable record ID, reservoir metadata, engineering

dimension or primary synthesis pattern, source IDs, EU IDs, KC IDs, locators, confidence, and evidence-depth fields.

5. Encode with a selected backend when configured. If no backend is configured, write structured text records that are index-ready.
6. Write index manifest with record counts, backend/model metadata, and build status.
7. Validate schema conformance and traceability integrity.

Outputs:

- 05_indexing/encoded_knowledge_records.jsonl
- 05_indexing/index_manifest.json
- 05_indexing/automated_validation.json
- 05_indexing/human_review.json
- 05_indexing/stage_manifest.json
- 05_indexing/stage_summary.md

Boundary:

Indexing is offline encoding. It does not retrieve records, answer user questions, create benchmark questions, or add new knowledge claims.

A.6 Report Generation Prompt

Task: Generate an evidence-grounded reservoir-operation report from validated records.

Inputs:

- runs/<run_id>/01_source_acquisition/sources/source_inventory.jsonl
- runs/<run_id>/02_evidence_extraction/evidence_units.jsonl
- runs/<run_id>/03_knowledge_consolidation/knowledge_cards.jsonl
- runs/<run_id>/04_synthesis/synthesis_cards.jsonl

Instructions:

1. Confirm validated EUs, KCs, SCs, and source inventory exist.
2. Select report type: technical report, short briefing, case-study summary, or validation report.
3. Build the report outline from validated EUs, KCs, and SCs.
4. Draft readable sections that distinguish source-grounded findings from synthesis-level interpretation.
5. Cite EU IDs, KC IDs, SC IDs, or source IDs for every material claim.
6. Explicitly mark uncertainty, evidence gaps, parser issues, source limitations, and unresolved issues.
7. Create a claim-evidence map for substantive claims.
8. Validate the report and claim-evidence map for schema conformance and traceability before final handoff.

Outputs:

- 06_report_generation/<report_name>.md
- 06_report_generation/claim_evidence_map.jsonl
- 06_report_generation/automated_validation.json

- 06_report_generation/human_review.json
- 06_report_generation/stage_manifest.json
- 06_report_generation/stage_summary.md

Boundary:

The report is a controlled rendering of validated records. It is not an independent evidence source and must not introduce unsupported claims.

Appendix B: Formal Notation and Schema Backup

This appendix keeps the more formal schema-style expressions that are useful for implementation, review, or supplementary material. The paper body avoids most of this notation to keep the method readable.

B.1 Run-Level Sets and Transformation

For one reservoir run, let:

$$\mathcal{S} = \{s_i\}_{i=1}^n$$

be the preserved source corpus. The workflow transforms sources into record layers and final artifacts:

$$\overset{f_E}{\mathcal{S}} \rightarrow \overset{f_K}{E} \rightarrow \overset{f_A}{K} \rightarrow \overset{f_I}{A} \rightarrow \overset{f_R}{I} \rightarrow R$$

where E is the Evidence Unit set, K the Knowledge Card set, A the Synthesis Card set, I the index-ready record set, and R the report plus claim-evidence map.

Each stage transition is gated by automated validation:

$$f_{t+1}(Y_t) \text{ is admissible only if } V_t(Y_t) = 1$$

with:

$$V_t(Y_t) = \mathbf{1} \left[\bigwedge_{g \in G_t} g(Y_t) = \text{pass} \right]$$

and:

$$G_t = \begin{cases} \{\text{SchemaConformance}, \text{AcquisitionSuccess}\} & t = 1 \\ \{\text{SchemaConformance}, \text{TraceabilityIntegrity}\} & t = 2, \dots, 6 \end{cases}$$

B.2 Traceability

Let $\text{Trace}(x)$ be the set of source-locator-quote triples that support record x :

$$\text{Trace}(x) = \{(s_i, l_i, q_i) : s_i \in \mathcal{S}, x \text{ is supported by locator } l_i \text{ and quotation } q_i \text{ in } s_i\}$$

A record is traceable when:

$$\text{Traceable}(x): \Leftrightarrow \text{Trace}(x) \neq \emptyset$$

The global layer constraint is:

$$\forall x \in (E \cup K \cup A), \quad \text{Traceable}(x)$$

B.3 Source Acquisition Schema

A source record can be represented as:

$$s_i = (id_i, title_i, url_i, F_i, D_i, Q_i, A_i, P_i, T_i, W_i)$$

where F_i is source family, D_i document type, Q_i content quality, A_i accessibility status, P_i local preservation paths, T_i extracted-text paths, and W_i warnings or recovery notes.

The selected source set is drawn from a screened candidate set $\mathcal{C}$:

$$\mathcal{S} = \{s_i \in \mathcal{C} : \text{Selected}(s_i) = 1 \wedge \text{PreservedOrRecorded}(s_i) = 1\}$$

For source-screening review, precision and recall may be summarized as:

$$\text{Precision}_{src} = \frac{|\mathcal{C}_{selected} \cap \mathcal{C}_{relevant}|}{|\mathcal{C}_{selected}|}, \quad \text{Recall}_{src} = \frac{|\mathcal{C}_{selected} \cap \mathcal{C}_{relevant}|}{|\mathcal{C}_{relevant}|}$$

These metrics evaluate source-selection decisions. They do not replace source adequacy review.

B.4 Evidence Unit Schema

Evidence extraction maps each preserved source into zero or more accepted EUs:

$$E = \bigcup_{s_i \in \mathcal{S}} f_E(s_i)$$

An EU can be represented as:

$$e_j = (id_j, s_i, l_j, q_j, d_j, m_j, c_j, n_j)$$

where id_j is the EU ID, s_i the source ID, l_j the locator, q_j the evidence quote, d_j the engineering dimension, m_j the finding text and why-it-matters note, c_j confidence, and n_j notes.

The acceptance predicate is:

$$\text{Accept}(e_j) = \mathbf{1}[\text{DocLevel}(e_j) \wedge \text{Operational}(e_j) \wedge \text{SelfContained}(e_j) \wedge \text{Traceable}(e_j)]$$

Only accepted EUs enter the consolidation input set.

B.5 Knowledge Card Schema

Knowledge consolidation groups EUs by operational question:

$$E_m = \{e_j \in E : Q(e_j) = Q_m\}$$

A Knowledge Card can be represented as:

$$k_m = (id_m, Q_m, E_m, d_m, T_m, F_m, N_m, c_m)$$

where id_m is the KC ID, Q_m the operational question, E_m the supporting EU set, d_m the engineering dimension, T_m the consolidation type, F_m the consolidated finding, N_m the source-coverage note, and c_m confidence.

The consolidation type must be one of:

$$T_m \in \{\text{repeated_fact, complementary_facts, contested_or_unresolved}\}$$

For different operational questions, card support sets remain separated unless the questions are intentionally merged by review:

$$Q_m \neq Q_{m'} \Rightarrow E_m \cap E_{m'} = \emptyset$$

B.6 Synthesis Card Schema

A Synthesis Card can be represented as:

$$a_r = (id_r, R_r, E_r, K_r, \tau_r, \sigma_r, C_r, U_r, O_r, \delta_r, c_r, v_r)$$

where id_r is the SC ID, R_r the synthesis claim, $E_r \subseteq E$ the supporting EU set, $K_r \subseteq K$ the relevant KC set, τ_r the primary pattern, σ_r the secondary lenses, C_r the analysis chain, U_r uncertainty or exception, O_r operational implication, δ_r evidence depth, c_r confidence, and v_r source verification context.

The primary pattern τ_r belongs to this controlled set: decision_process, constraint_structure, operational_tradeoff, operating_regime_change, historical_operation_failure, or operational_consequence.

Each card carries exactly one primary pattern:

$$|\{\tau_r\}| = 1$$

Secondary lenses must not duplicate the primary pattern:

$$\tau_r \notin \sigma_r$$

For the analysis chain:

$$C_r = \{p_{r,1}, \dots, p_{r,w}\}, \quad w \geq 2$$

Each step must cite at least one supporting EU:

$$\forall p_{r,u} \in C_r, \quad \exists e_j \in E_r: \text{Grounded}(p_{r,u}, e_j) = 1$$

where:

$$\text{Grounded}(p, e_j) = \mathbf{1}[e_j \in E_p]$$

and E_p is the EU set explicitly cited by step p .

For source-checked cards:

$$\delta_r = \text{source_checked} \Rightarrow v_r = (\text{source_locator_summary}, \text{source_verification_note})$$

B.7 Index Record Schema

Each index record wraps exactly one EU, KC, or SC:

$$I = \{l_u = (id_u, z_u, \text{text}_u, M_u) : z_u \in (E \cup K \cup A)\}$$

where id_u is the index record ID, z_u the underlying record, text_u the index-ready text, and M_u metadata including record type, reservoir ID, source IDs, EU IDs, KC IDs, confidence, pattern or dimension, and encoding metadata.

Index records inherit traceability:

$$\text{Trace}(l_u) := \text{Trace}(z_u)$$

Retrieval evaluation, when added, can use query-level precision and recall:

$$\text{Precision}_{@k}(q) = \frac{1}{k} \sum_{i=1}^k \text{rel}(r_i, q), \quad \text{Recall}_{@k}(q) = \frac{\sum_{i=1}^k \text{rel}(r_i, q)}{|\mathcal{R}_q|}$$

where r_i is the record at rank i , $\text{rel}(r_i, q)$ indicates whether the record is relevant to query q , and $\mathcal{R}_q$ is the review-defined relevant record set.

B.8 Report and Claim-Evidence Map Schema

The report output is:

$$R = (D, P)$$

where D is the report document and P is the claim-evidence map:

$$P = \{p_v = (id_v, \pi_v, Z_v, \lambda_v)\}$$

Here id_v is the claim ID, π_v the claim text, $Z_v \subseteq (\mathcal{S} \cup E \cup K \cup A)$ the cited evidence set, and λ_v the support level.

A report claim is admissible only when it cites at least one preserved source or traceable record:

$$\forall p_v \in P, \quad Z_v \neq \emptyset \wedge \forall z \in Z_v, [z \in \mathcal{S} \vee \text{Traceable}(z)]$$

The report is therefore a controlled rendering of validated records, not an independent source of reservoir-operation knowledge.

B.9 Human Review Scores

For a sampled human-review criterion h , the normalized stage score can be written as:

$$HR_{t,h} = \frac{1}{2N_{t,h}} \sum_{i=1}^{N_{t,h}} score_{i,h}, \quad score_{i,h} \in \{0, 1, 2\}$$

The normalized score is useful for summary figures, while the original 0/1/2 decisions remain the review record.